

Prova Frontend 2025 7 páginas.

Parte 1: React e React Native (6 questões - 20 pontos)

1. (2 pontos) Em React, qual gancho é utilizado para gerenciar estado em componentes funcionais?
 - a) useEffect
 - b) useState
 - c) useContext
 - d) useReducer
2. (2 pontos) Quais das seguintes bibliotecas são utilizadas para navegação no React Native?
 - a) React Navegação
 - b) React Router
 - c) NavigationJS
 - d) RouterNative
3. (2 pontos) Qual a melhor forma de evitar re-renderizações desnecessárias em componentes funcionais no React?
 - a) Usar o gancho useMemo
 - b) Usar o gancho useState
 - c) Declarar as variáveis como globais
 - d) Usar o método componentDidMount
4. (5 pontos) No React Native, como você estilizou componentes para que ocupem 50% da largura da tela?
 - a) width: "50%"
 - b) flex: 0,5
 - c) width: Dimensions.get("janela").width / 2
 - d) width: 50
5. (5 pontos) Qual a principal vantagem de usar a API de Contexto em React?
 - a) Substituir Redux e gerenciar estado global sem bibliotecas externas
 - b) Permitir a navegação entre componentes
 - c) Melhorar o desempenho do código
 - d) Reduzir o tamanho do DOM virtual
6. (4 pontos) Qual diferença fundamental existe entre React e React Native?
 - a) React usa CSS nativo enquanto React Native usa folhas de estilo.
 - b) React é usado para web e React Native é usado para dispositivos móveis.
 - c) React Native não suporta JSX.
 - d) React não suporta ganchos, mas React Native sim.

Parte 2: JavaScript (8 questões - 10 pontos)

7. (1 pontos) O que significa typeof null em JavaScript?
 - a) "nulo"
 - b) "objeto"
 - c) "indefinido"
 - d) "função"
8. (1 pontos) Qual será o resultado da seguinte expressão?
console.log(0.1 + 0.2 === 0.3);
 - a) verdadeiro
 - b) falso
 - c) indefinido
 - d) NaN

9. (1 pontos) Qual é a saída de: `console.log([] + {});`
- a) "[objeto Objeto]"
 - b) "[objeto Objeto][]"
 - c) "indefinido"
 - d) Um erro será lançado
10. (1,5 pontos) Como declarar uma função anônima em JavaScript?
- a) `function() {}`
 - b) `() => {}`
 - c) `function nome() {}`
 - d) Ambas as opções a e b estão corretas
11. (1,5 pontos) Qual a diferença entre `var`, `let` e `const`?
- a) `var` é escopo de função, enquanto `let` e `const` são escopos de bloco.
 - b) `var` e `let` são mutáveis, enquanto `const` não é.
 - c) `let` é mais rápido que `var`.
 - d) Apenas `let` permite criar arrays.
12. (1,5 pontos) Qual é o resultado do próximo código?
- ```
let x = "5";
let y = 2;
console.log(x * y);
```
- a) 10
  - b) NaN
  - c) 52
  - d) indefinido
13. (1,5 pontos) O que significa o operador `??` em JavaScript?
- a) Operador de nulidade
  - b) Operador lógico "ou"
  - c) Operador ternário
  - d) Operador de coalescência nula
14. (1 pontos) Como você cria uma promessa em JavaScript?
- a) `const p = new Promise((resolve, reject) => { /* lógica */ })`
  - b) `const p = Promise.resolve()`
  - c) `const p = async () => { return ... }`
  - d) Todas as opções acima

### Parte 3: TypeScript (8 questões - 20 pontos)

15. (2 pontos) Como definir uma variável de tipo explícito em TypeScript?
- a) `let x: number = 5;`
  - b) `let x = number 5;`
  - c) `let x int = 5;`
  - d) `let number x = 5;`
16. (2 pontos) Qual é a função principal de inferência de tipos em TypeScript?
- a) Permitir usar JavaScript puro sem alterações.
  - b) Evitar a necessidade de declaração do tipo manualmente.
  - c) Permitir tipagem dinâmica no código.
  - d) Garantir que não haja erros de sintaxe.
17. (3 pontos) Como declarar uma função genérica em TypeScript?
- a) `function(arg: T): T {}`
  - b) `function(arg: T) {}`
  - c) `function(T arg): T {}`
  - d) `function(arg): T {}`

18. (3 pontos) Qual é o propósito do `unknown` em TypeScript?
- a) Indicar um tipo indefinido.
  - b) Substituir `any` com maior controle.
  - c) Indicar um valor que será inicializado futuramente.
  - d) Tipo de objeto desconhecido.
19. (3 pontos) Quais das opções abaixo são verdadeiras sobre interfaces e tipos em TypeScript?
- a) Interfaces e tipos são intercambiáveis em qualquer contexto.
  - b) Interfaces permitem herança múltipla, enquanto tipos não.
  - c) Tipos autorizam união de declarações, enquanto interfaces não.
  - d) Tanto b quanto c estão corretas.
20. (2 pontos) Dado o código abaixo, qual será o erro?
- ```
let value: number | string = 10;
value.toUpperCase();
```
- a) Nenhum erro será lançado.
 - b) Erro de tempo de execução.
 - c) Erro de compilação porque `toUpperCase` não existe no tipo `number`.
 - d) Erro de compilação porque `value` não é um número.
21. (3 pontos) Qual a diferença entre `readonly` e `const` em TypeScript?
- a) `readonly` é usado em objetos e propriedades, enquanto `const` é usado para variáveis.
 - b) `readonly` é mais performático que `const`.
 - c) `const` só funciona para números, enquanto `readonly` funciona para strings.
 - d) Não há diferença significativa.
22. (3 pontos) Como declarar um parâmetro opcional em TypeScript?
- a) `function example(param?: string)`
 - b) `function example(param: string | null)`
 - c) `function example(param?: any)`
 - d) Ambas as opções a e c estão corretas.

Parte 4: Lógica de Programação (8 questões - 20 pontos)

23. (4 pontos) Qual é a saída do seguinte código?
- ```
const x = [1, 2, 3];
x[10] = 50;
console.log(x.length);
```
- a) 11
  - b) 10
  - c) 3
  - d) indefinido
24. (4 pontos) Dado um array `[1, 3, 5, 7]`, como você obteria o valor 5?
- a) `array[1]`
  - b) `array[2]`
  - c) `array[3]`
  - d) `array.indexOf(5)`
25. (3 pontos) Em pseudocódigo, qual algoritmo representa uma busca binária?
- a) Dividir o array ao meio e procurar no lado correto.
  - b) Procurar sequencialmente até encontrar.
  - c) Ordenar o array e verificar o índice mais alto.
  - d) Comparar cada elemento com o próximo.

26. (3 pontos) Qual será o resultado de: `console.log("5" + 5 - 5);`
- a) 10
  - b) 5
  - c) 50
  - d) NaN
27. (4 pontos) Quantos loops for são necessários para percorrer todos os elementos de uma matriz 2D?
- a) 1
  - b) 2
  - c) 3
  - d) Depende do tamanho da matriz
28. (2 pontos) Qual será o resultado de `typeof NaN`?
- a) number
  - b) undefined
  - c) null
  - d) string

## Parte 5: Clean Code e KISS (9 questões - 10 pontos)

29. (2 ponto) Dado pseudocódigo:

```
if (x > 10) {
 return "maior";
} else {
 return "menor";
}
```

Qual é a refatoração seguindo o KISS?

- a) `return x > 10 ? "maior" : "menor";`
- b) `return x >= 10 ? "maior" : "menor";`
- c) `if (x > 10) return "maior"; else return "menor";`
- d) `return x == 10 ? "igual" : "menor";`

30. (1 ponto) Quais os seguintes nomes de variáveis seguem as práticas de Código Limpo?

- a) x
- b) calculateSum
- c) sumCalc
- d) calsum

31. (1 ponto) O que é considerado código "limpo" em Clean Code?

- a) Código funcional, mesmo que difícil de entender.
- b) Código que qualquer desenvolvedor possa ler e entender rapidamente.
- c) Código com muitos comentários explicativos.
- d) Código com poucas linhas, mesmo que complexo.

32. (1 ponto) Qual é o problema de funções que fazem mais de uma coisa?

- a) São difíceis de testar e manter.
- b) Gastam mais memória.
- c) Tornam o código mais performático.
- d) Não tenham problemas, desde que sejam curtas.

33. (1 ponto) Qual das opções abaixo não é uma prática de Código Limpo?

- a) Usar nomes descritivos para variáveis e funções.
- b) Criar funções pequenas e específicas.
- c) Adicione muitos comentários em código claro.
- d) Remover código morto ou desnecessário.

34. (1 ponto) O que o princípio DRY (Don't Repeat Yourself) sugere?
- a) Evitar duplicação de lógica ou dados no código.
  - b) Manter o código curto e sem comentários.
  - c) Reutilizar funções sempre que possível.
  - d) Criar funções que sejam aceites em intervalos múltiplos.
35. (1 ponto) O que significa refatorar o código?
- a) Reescrever partes do código sem alterar sua funcionalidade.
  - b) Adicionar novas funcionalidades ao código.
  - c) Trocar nomes de variáveis.
  - d) Melhorar o desempenho de algoritmos.
36. (1 ponto) Qual é o objetivo principal de seguir o princípio YAGNI (You Are not Gonna Need It)?
- a) Implementar funcionalidades futuras para evitar retrabalho.
  - b) Evitar implementar funcionalidades desnecessárias antes que sejam realmente necessárias.
  - c) Criar um código genérico que possa ser reutilizado.
  - d) Planejar antecipadamente todas as possibilidades do projeto.
37. (1 ponto) Qual é o problema de adicionar muitos comentários ao código?
- a) Os comentários podem ficar desatualizados e gerar confusão.
  - b) Comentários deixam o código mais lento.
  - c) Comentários aumentam o tamanho do arquivo.
  - d) Comentários impedem o uso de boas práticas.

## Parte 6: Programação Orientada a Objetos (10 questões - 20 pontos)

38. (2 pontos) Qual dos itens abaixo é considerado um dos pilares do POO?
- a) Abstração, Polimorfismo, Modularidade
  - b) Encapsulação, Herança, Polimorfismo
  - c) Classes, Objetos, Funções
  - d) Encapsulação, Modularidade, Iteração.
39. (2 pontos) O que é encapsulamento no POO?
- a) A capacidade de ocultar os detalhes internos de implementação de uma classe.
  - b) O ato de criar instâncias de uma classe.
  - c) A herança de propriedades de uma classe para outra.
  - d) A capacidade de reutilizar código em métodos privados.
40. (2 pontos) Qual palavra-chave é usada para declarar um método que deve ser implementado por classes filhas em TypeScript?
- a) abstract
  - b) interface
  - c) protected
  - d) implements
41. (2 pontos) Qual é a principal diferença entre *private* e *protected* em POO?
- a) *private* permite acesso apenas dentro da classe, enquanto *protected* permite acesso também em classes derivadas.
  - b) *private* e *protected* são sinônimos.
  - c) *protected* permite acesso global, enquanto *private* restringe o acesso a um módulo.
  - d) Não há diferença entre *private* e *protected*.
42. (2 pontos) O que significa sobrescrita (override) de métodos?
- a) Alterar o comportamento de um método em uma classe filha.
  - b) Declarar métodos múltiplos com o mesmo nome na mesma classe.
  - c) Reutilizar métodos de classes externas.
  - d) Declarar métodos privados em uma classe pai.
43. (2 pontos) Qual será o resultado do próximo código em TypeScript?

```
class Animal {
 makeSound(): string {
 return "Som genérico";
 }
}
class Cachorro extends Animal {
 makeSound(): string {
 return "Latido";
 }
}
```

```
const a: Animal = new Cachorro();
console.log(a.makeSound());
```

- a) "Som genérico"
- b) "Latido"
- c) Um erro será lançado na execução.
- d) Indefinido

44. (2 pontos) Qual é o termo usado para criar métodos múltiplos com o mesmo nome, mas com assinaturas diferentes?

- a) Sobrecarga
- b) Sobrescrita
- c) Herança múltipla
- d) Polimorfismo estático

45. (2 pontos) Como se chama a prática de ocultar a implementação interna de uma classe e expor apenas o que é relevante para o usuário?

- a) Abstração
- b) Polimorfismo
- c) Modularidade
- d) Sobrecarga

46. (2 pontos) Qual o resultado do código a seguir?

```
class A {
 private myValue = -10;
 private value = 10;
 getValor(): number {
 return this.valor;
 }
}
class B extends A {
 private myValue = 20;
 private value = 10
 getValor(): number {
 return super.getValor() + this.myValue;
 }
}
```

```
const b = new B();
console.log(b.getValor());
```

- a) 10
- b) 20
- c) 30
- d) Um erro será lançado.

47. (2 pontos) O que significa o termo “composição” no POO?

- a) Uma classe que reutiliza métodos de outra classe através de herança.
- b) Uma classe que é composta por várias funções estáticas.
- c) Uma classe que utiliza objetos de outra classe para compor seu comportamento.
- d) Uma classe que só pode ser instanciada por uma turma filha.